

12.1.1. Phone Number Database Management

Algorithm: Phone Number Database using Dictionary

1. **Start**
 2. **Input number of operations**
 - Read integer n
 3. **Initialize dictionary**
 - Create an empty dictionary: contacts = {}
 4. **Repeat for n operations**
 - For each operation:
 1. Read input and split into parts → operation[]
 2. Check the first element of operation:
-

Case 1: ADD operation

- If operation[0] == "ADD":
 1. Extract name = operation[1]
 2. Extract phone = operation[2]
 3. Add or update in dictionary:
 - contacts[name] = phone
-

Case 2: REMOVE operation

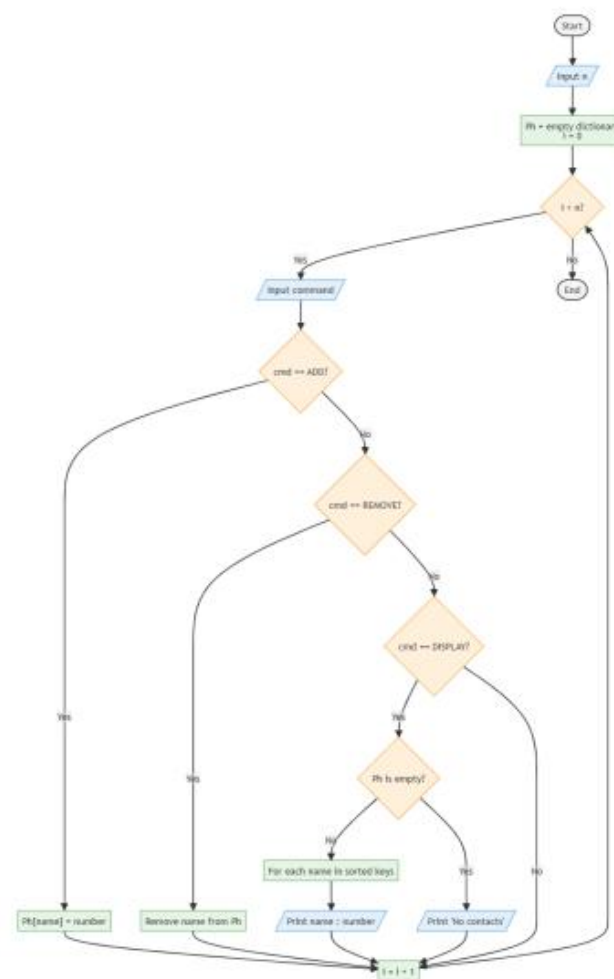
- If operation[0] == "REMOVE":
 1. Extract name = operation[1]
 2. If name exists in dictionary:
 - Delete it → del contacts[name]
 3. Else:
 - Do nothing (ignore)
-

Case 3: DISPLAY operation

- If operation[0] == "DISPLAY":
 1. If dictionary is empty:
 - Print "No contacts"
 2. Else:
 - Sort keys alphabetically
 - For each name in sorted order:
 - Print → name: phone_number

5. **End loop**

6. **Stop**



Flowchart :

Programme :

12.1.1. Phone Number Database Management

Write a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values.

The program should support three operations: adding a contact (if the name already exists, update the phone number), removing a contact (if the name does not exist, ignore the operation), and displaying all contacts in alphabetical order by name.

Input Format:
1. First line contains an integer n representing the number of operations.
2. Each of the next n lines contains a single operation in one of the following formats:

- "ADD <name> <phone_number>" where <name> is a string, <phone_number> is a string of 10 digits.
- "REMOVE <name>" - name is a string.
- "DISPLAY" - no additional parameters.

Output Format:

- For display operation, print all contacts in alphabetical order by name in the format:
<name> : <phone_number>
- If the database is empty during display operation, print "No contacts".
- Do not print anything for add and remove operations.

Constraints:

- Name contains only alphabetic characters (no spaces). And it is case sensitive.

Sample Test Cases

phoneDat...

```
1 n = int(input())
2 contacts = {}
3
4
5 for _ in range(n):
6     operation = input().split()
7
8     if operation[0] == "ADD":
9         name = operation[1]
10        phone = operation[2]
11        contacts[name] = phone
12
13    elif operation[0] == "REMOVE":
14        name = operation[1]
15        if name in contacts:
16            del contacts[name]
17
18    elif operation[0] == "DISPLAY":
19        if not contacts:
```

Average time: 0.023 s, Maximum time: 0.031 s
5 out of 5 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test Case 1
Expected output:
ADD Alice: 9876543210
ADD Bob: 8765432109
ADD Charlie: 7777777777
DISPLAY
Alice: 9876543210
Actual output:
4
ADD Alice: 9876543210
ADD Bob: 8765432109
ADD Charlie: 7777777777
DISPLAY
Alice: 9876543210